

- Basic Architecture
- Operation
- Programmer's View
- Development Environment
- Application-Specific Instruction Set Processors
- Selecting a Microprocessor
- General-Purpose Processor Design

Basic Architecture

A general-purpose processor is a programmable digital system which consists of a datapath and a controller which are tightly linked with a memory. Figure 3.1 shows the various components in the architecture of the general-purpose processor.

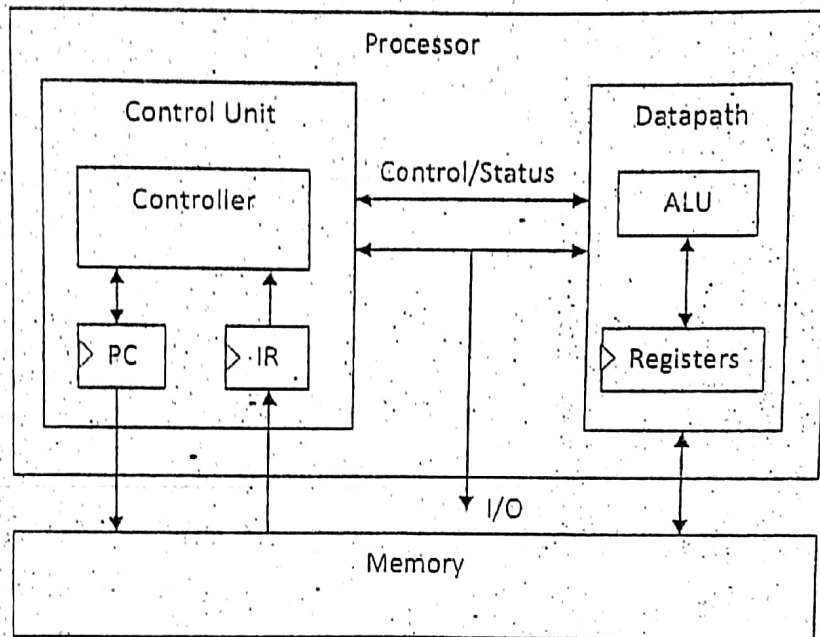


Figure 3.1: Basic Architecture of general-purpose processor

DATAPATH

Datapath consists of the circuitry for transforming data and for temporary data storage. It contains an arithmetic-logic unit which manipulates data through various operations such as addition, subtraction, logical AND, logical OR, rotating, shifting etc. ALU also generates status signals to represent various conditions such as carry, zero, sign, parity and so on. Such information is stored in status register.

Data path contains registers to store temporary data and different status generated by operations. The temporary data may be the data from memory for ALU to process, or the data that needs to be moved from one memory to another memory, or the data from ALU that needs further processing by ALU or needed storage. For data transfer within datapath, internal bus is used. But movement of data from and to memory is done by external bus.

CONTROL UNIT

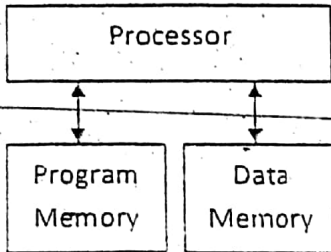
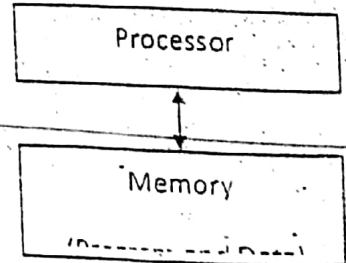
The control unit consists of circuitry to generate control signals to carry out various operations. It consists of controller, Program Counter (PC) and Instruction Register (IR). Controller consists of a state register and control logic. It sequences through the states and generates the control signals to read instructions into the Instruction Register, and control the flow of data between ALU, registers of datapath and memory. Controller also determines the next value of Program Counter. For non-branch instruction, the value of Program Counter is incremented. But for branch instruction, status signals from datapath and content of Instruction Register are evaluated for next address of program counter.

Program Counter is used to hold the address of the next program instruction to be fetched, while an Instruction Register is used to hold the fetched instruction. The bit width of Program Counter indicates the address size of memory which in turn can be used to determine the number of directly accessible memory locations. For example, A 16 bit PC represents address size of 16 bit and $2^{16} = 65536$ addressable memory locations.

MEMORY

Memory is used to store information for medium or long term. Information can be data or program. Program information is the set of instructions that is used to carry out desired function. Data are the information used by the program for various purposes.

There are two memory architectures based on program and data storage.

SN	Harvard Architecture	Princeton Architecture
1.	Distinct data and program memory space	Data and program share memory space
2.	Improved performance: Data and instructions can be fetched simultaneously	Data and instructions cannot be fetched simultaneously
3.	More connecting wires	Less connecting wires
4.	Block Diagram  <pre> graph TD P[Processor] <--> PM[Program Memory] P <--> DM[Data Memory] </pre>	Block Diagram  <pre> graph TD P[Processor] <--> M[Memory] </pre>

Operation

Instruction Execution

Instructions are the sets of code that carry out particular function. For each instruction, the controller sequences through several stages. Each stage may consist of one or more clock cycles. The various stages or sub-operations can be explained as:

- **Fetch Instruction:** The next instruction to be executed is loaded into Instruction Register from memory. The address of the memory where instruction resides is given by program counter.
- **Decode Instruction:** Instruction in the instruction register may represent various operations based on op-code and may include register or memory as operands. In this stage, the operation to be done by the instruction is determined.
- **Fetch Operand:** For a given operation, operand can be a register or memory. In operations including registers, the required data are loaded into registers as specified by the instruction.
- **Execute Operation:** The ALU handles the arithmetic and logical operations defined by the instructions. The loaded registers are fed to the inputs of ALU to carry out the operation.
- **Store results:** The destination to store results may be either register or memory. After the execution of operation, the final data is loaded into register or memory as defined by the instruction.

Pipelining

Pipelining is implemented to increase the throughput of the system. In pipeline, the given task is divided into various stages and multiple stages which are independent of each other are executed simultaneously. For efficient instruction pipeline, different stages must be of almost same length and each instruction must require same number of cycles to complete its execution.

Branching instructions can be an obstacle for efficient pipeline as next instruction to be executed will only be known after execution stage of branch instruction. This problem, however, can be addressed using various techniques. One simple method is to stall the pipeline when there is an occurrence of branching instruction. The pre-fetch of next instructions is not done in this method rather waited for execute stage to complete first. Another popular method is to use

branch prediction. In this method, the branch is guessed and the next instruction is fetched correspondingly. If the guess is correct, then it results in efficient pipeline. But, however, if the prediction is not correct then all pre-fetched instructions in the pipeline must be ignored. The following diagram shows an example of an instruction pipeline having five stages:

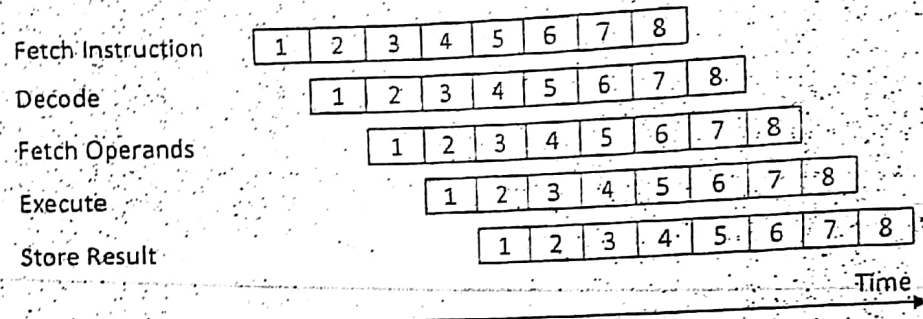


Figure 3.2: Eight instructions in execution using instruction pipeline

In figure 3.2, there are 8 instructions in the pipeline with each instruction divided into five stages and each requiring equal time to complete. In absence of pipeline, the total time required to complete eight instructions would be $8 \times 5 = 40$ clock cycles, assuming each stage to complete in one cycle. However, with pipeline implementation, the total completion time required is 12 clock cycles. In this way, pipeline helps to improve the performance of the system.

Superscalar and Very Long Instruction Word (VLIW) Architectures

Multiple ALU architecture is implemented in superscalar architectures to improve the performance of the system. Such systems can execute two or more scalar operations in parallel, which increase the requirement of ALU in the processor. It may require extensive hardware to detect multiple independent instructions that can be executed simultaneously. Instructions in such architectural systems are ordered statically (at compile time) or dynamically (during runtime).

Very Long Instruction Word (VLIW) architecture is a type of static superscalar architecture. It contains multiple independent instructions in a single word. Several operations are encoded in a single machine instruction. The compiler detects and schedules the instructions.

Programmer's View

Software programmer does not require detail understanding of architecture of the system; instead they need to know what instructions are available and how they are used. Embedded system programmer, however, needs to know certain information, if not all, about the system, as the programs may include assembly level language.

Instructions in the program may be of different level. Firstly, in machine level the codes are in binary form. Another level is assembly level in which mnemonics are used to represent instructions which are processor specific in nature. Next level of programs follow structured languages which are processor independent. Programmer of assembly language and machine language must have information about architecture of the processor.

In embedded system design, the programmer must be aware of the following:

A. Instruction Set

The instruction set is a list of instructions which represent the bit configurations for operations that can be carried out by the processor. Assembly language programmer must be aware of the available instruction set. Since embedded system design may require some portion of assembly code to be written, programmer of embedded system must know the instruction set available for the processor they are working on.

Every instruction, in general, consists of op-code and operand field. Op-code field specifies the operation to be done. Different types of instructions are explained briefly.

- Data-transfer instructions move data from memory to register, register to memory or register or input/output ports.
- Arithmetic and logical instructions cause ALU to carry out certain operations involving registers and store the final result back to register. ADD, SUB, AND, OR etc are few examples of arithmetic and logical operations.
- Branch instructions change the flow of program and it also determines the address of the next instruction to be executed. Branch instructions may be unconditional jumps, conditional jumps or procedure call and return instructions.

An operand field specifies the location of actual data that takes part in an operation. The number of operands per instructions varies among processors and its instruction type. Addressing modes are used to represent data location and its accessing mechanism. The simple instruction format is shown in the figure below.

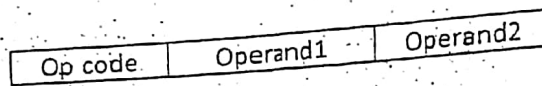


Figure 3.3: Simple two address instruction format

Commonly used addressing modes are explained in the following paragraph.

Immediate Addressing Mode: The operand field contains the actual data.

Register Direct Addressing Mode: The operand field contains the address of the datapath register in which the data is stored.

Register Indirect Addressing Mode: The operand field contains the address of a register, which in turn contains the effective address of the data in memory.

Direct Addressing Mode: The operand field contains the effective address of actual data that is used in operation.

Indirect Addressing Mode: The operand field contains the address of a memory location, which in turn contains the address of a memory location in which actual data is available.

Implicit or Implied Addressing Mode: The operand field is not used in this mode; the register to be used in operation is defined implicitly. In general, accumulator is used as an implicit register.

Displacement Addressing Mode: The operand is added to a particular register to obtain the effective address of the data. In index addressing, index registers are used. While in relative addressing, value of operand is added to the current address to determine the actual address.

The operations of few addressing modes can be visualized using following figure.

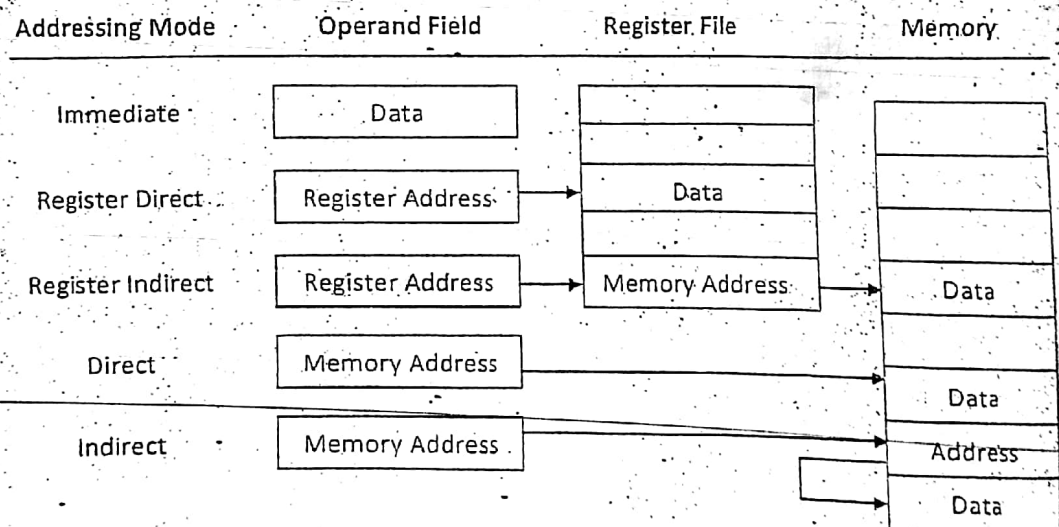


Figure 3.4: Addressing Modes

B. Program and Data Memory Space

The programmer in embedded system design must be aware of the size available for program and for data. Programs must be written within the defined memory space limits. For example, in microcontrollers, the on-chip memory for program and data are fixed. So, one should be able to write the code efficiently so as not to exceed the memory limit.

C. Available Registers

Programmer of embedded system design must be informed about the number of registers available for general purpose and specific purpose. For example, multiplication in 8051 microcontroller can be done using accumulator and register B. Information about accumulator and Register B is not required for structured language programmer. However, various special function registers used for configuring timers, serial communication, and interrupts must be known to every programmer.

D. Input Output Facility

Every processor facilitates programmer with input output pins to communicate with external devices. Programmer working with processors must be alert about the number of input output pins available and their functions. In parallel I/O, port can be read or written to using specific function register. Also, communications can be done through system bus in which address and data ports can be activated by certain instructions.

E. Interrupts

Interrupt is a facility provided to the user in which the processor serves the device which requires urgent attention. It causes processor to suspend execution of the current program and starts executing interrupt service routine that does the function required by the device which interrupts the processor. The programmer should be aware of the types of interrupts supported by the processor and must write interrupt service routine when required.

F. Operating System

An operating system is a layer of software that provides low-level services to the application layer. Few services involve loading and executing of programs, sharing and allocating system resources, and synchronization mechanism. Another important service is process scheduling in which the high priority process is executed first. Other services include handling hardware interrupts, and provide device drivers.

High level applications invoke operating system using system call. When a program requires service from operating system, it generates a predefined software interrupt that is served by the operating system. Values required to the services are typically passed as the parameters in the program. CPU registers are involved for information exchange among application programs and operating system.

3.4 Development Environment

Processors along with different development tools are used for the development of software or an embedded system. Processor that is used to write and debug the program is commonly referred as development processor. Desktop Computer can be taken as an example of development processor. Such processors may not be a part of embedded system's implementations. But the processor in which our program is loaded is referred as target processor. AVR, 8051, PIC microcontrollers or 8085, 8086 microprocessor can be few examples of target processor. Such processors are always a part of system implementations. Various tools for the software development as well as embedded systems development are described in the following paragraphs:

Tools for Implementation Phase

Assemblers convert assembly instructions to binary machine instructions. It replaces op-code and operand mnemonics by binary equivalent. It also translates symbolic labels into actual addresses. It generates a equivalent binary code for a single machine instruction, so it follows one to one mapping principle.

Compilers convert high level programs to machine programs. Each high-level constructs may be translated to several machine instructions. Hence, it may not follow one to one mapping principle. Cross compilers are those compilers which run on one processor but generate the code for a different processor.

Linker combines object files into a single executable file, or another object file. It allows creation of a program in separately assembled or compiled files. It combines machine instructions of user code and instructions from standard library.

Tools for Verification Phase

Debuggers are programs that are used to test and debug the targeted program. These are programs that run on development processor but execute code designed for target processors. It simulates the function of the target processors and allows evaluation and correction of

programs in development processor. Stepping, breakpoints, watch values are few debugging techniques supported by various debuggers. These debuggers are also known as instruction set simulators (ISS) or virtual machines (VM). Design cycle for debuggers is fast as compared to other tools, since the program is tested in development processor. But, these tools can, however, lead to inaccuracy as it does not interact with the actual system.

Emulator can be a hardware or software that enables one system to behave like another system. It consists of debugger coupled with a board connected to development processor. The board consists of target processor or device similar to target processor and support circuitry. It supports debugging of program while it executes on target processor. It also enables one to control and monitor the program's execution in actual embedded system circuit. Since the code must be downloaded into emulator hardware in each test, the design cycle is little longer compared to debugger. But it leads to accurate testing as it interacts with the rest of the system components as well.

Device Programmers are the devices with the help of which binary machine programs are loaded into target processor's memory. Using this tool, the program can be tested in its realistic form which results in high accuracy as program runs on actual system. The design cycle, however, is longest since the target processor is removed from the system, programmed using programmer and returned to the system. If the device programmer can be made in-build within the system, the design cycle will be reduced.

DESIGN FLOW

Every software or system development process includes implementation and verification phase. During implementation phase, various implementation tools such as assembler, compiler are used while verification tools such as debugger, programmer are used in verification phase.

Software Development Process

For a software development, the development processor as well as the target processor may be common. And the development tools are available in a single package which is referred as Integrated Development Environment (IDE).

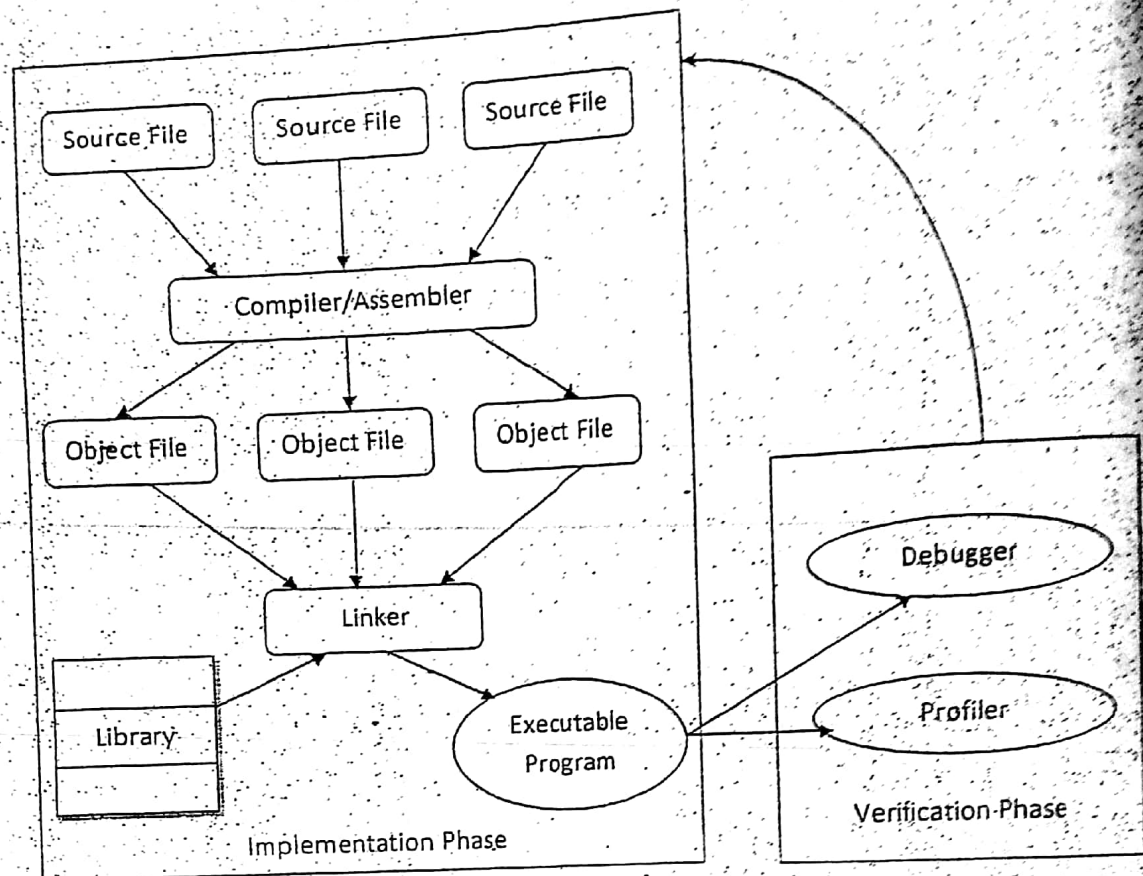


Figure 3.5: Software Development Process

Implementation Phase: Source code is written using an editor, and then the code is compiled/assembled using compiler/assembler. Finally, with the help of linker all required files are combined into a final executable file.

Verification Phase: The executable file is run under the command of a debugger. All possible inputs, especially boundary cases, are used to check the behavior of program. Profilers can be used for performance analysis of the program. Time and space complexity can be analyzed. Time complexity includes duration of execution of program whereas space complexity includes memory usage.

Embedded System Development Process

In case of embedded system design, the target and development processors are different in almost all systems. The Integrated Development Environment (IDE) tools for various processors are available for implementation phase. Though the implementation phase for embedded

system is similar to that of software implementation phase, the verification phase differs drastically.

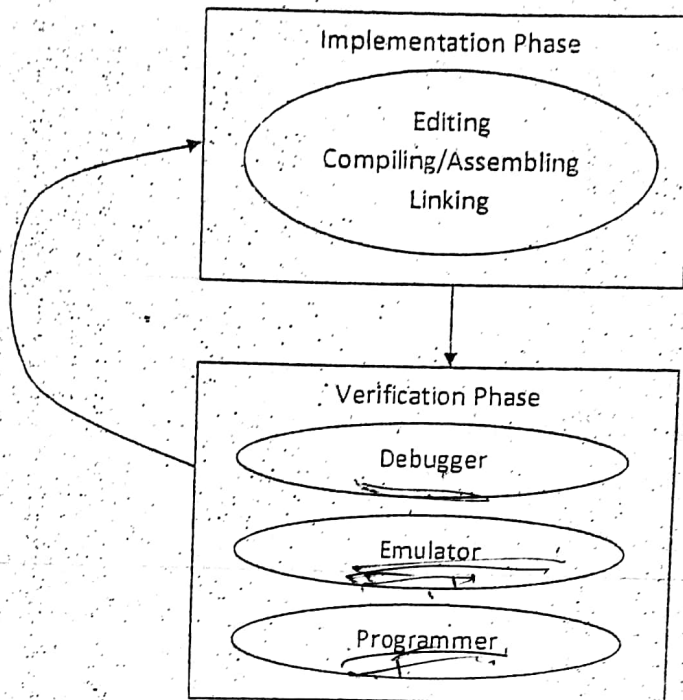


Figure 3.6: Embedded System Development Process

Implementation Phase: The process of editing, compiling/assembling and linking the program is same as that of software development process. However, development processors use cross compilers or cross assembler. As those compilers run on development processor, for example PC, and generate the file for target processor, for example hex file for microcontrollers.

Verification Phase: Embedded system works in conjunction with other components as well as with real time environment, so debugging a program requires control over time and environment. Based on requirement and availability, debuggers, emulators or device programmers can be used for verification. Code may be simulated on development processor using debuggers or code may be checked by loading into emulator hardware. Also, programmer can be used to load the code directly into the target processor.

3.5 Application-Specific Instruction Set Processors

Application-Specific Instruction Set processors are specific to the particular application domain. They can be programmed based on requirement of particular arena, which makes it more flexible. Also other constraints such as performance, power, cost, and size are efficient enough to develop a system. However, instruction set processor and its associated software tools are expensive to develop. It can be categorized as microcontrollers, digital signal processors and less general application specific instruction set processors.

Microcontrollers

Microcontrollers are specific to applications that perform a large amount of control oriented tasks. The following are few general features of microcontrollers.

- It includes several peripheral devices such as timers, analog to digital converters, serial communication devices, and so on.
- It generally contains program and data memory on the same IC. Various peripherals along with memory incorporated within the same IC result in compact and low-power implementation.
- It provides the programmer direct access to number of pins of the IC. Access to pins enable programmer to interface with other devices such as sensor, actuators, LCDs, and other different devices that may be used in the system.
- Some specialized instructions may be available. Such facility improves the performance of the system.

Digital Signal Processors (DSP)

These are processors which are specific to applications that process large amounts of data. The source of large amount of data includes image captured by a camera, voice packet through a network router, audio clip played by an instrument. Few features, out of many, are listed below.

- It may contain numerous register files, memory blocks, multipliers and other arithmetic units.
- It facilitates with instructions that are applicable uniquely to digital signal processing. Filtering and transforming vectors can be two examples.

- Frequently used arithmetic functions are implemented using hardware. It results in faster execution of arithmetic functions compared to software implementation.
- Some special digital signal processors allow concurrent execution of functions which boost the performance of the system.
- It incorporates many peripherals specific to signal processing. It may include ADC, DAC, PWM, DMA controllers, Timers and Counters.

Less-General ASIP

These are developed to perform some very domain specific processing while allowing some degree of programmability. Processors designed for networking hardware can be taken as an example of less-general ASIP.

Selecting a Microprocessor

In any embedded system, a designer must select the microprocessor based on technical and nontechnical aspects.

- Technical aspects: Selection of processor must be done based on required speed within limited power, size, and cost.
- Non-technical aspects: Before selecting microprocessor, one must be aware of development environment, prior expertise of processor, licensing arrangements and so on.

Comparing Speed

Speed of processors can be measured and compared using various methods.

A. Clock Speed of Processor

Speed can be compared based on clock speeds of processors, but the number of instructions per clock cycle may differ. So, it may not be a efficient method unless processors to be compared have same number of instructions per cycle.

B. Instruction per second

The speed can be evaluated using number of instructions executed per second. But the complexity of available instruction sets may differ creating some hindrance in speed

comparison. For example, to perform same operation, one processor may require 20 instructions while another may require 300 instructions.

C. Dhrystone benchmark

It is a program that runs on different processors and evaluates their performance based on execution of certain operations. Dhrystone benchmark performs no useful work rather checks the integer arithmetic and string-handling capabilities of the processor on which the benchmark runs on. Since processors can execute such operations thousands of times in a second, speed of processor may be expressed in terms of Dhrystones per seconds.

D. Millions of Instruction Per Second (MIPS)

It is a general measure of computing performance and the amount of work a processor can do. MIPS can be useful when comparing performance of processors having similar architecture. The origin of MIPS is based on VAX 11/780 which could execute one million instructions per second or could execute 1757 Dhrystones per second. Hence, $1 \text{ MIPS} = 1757 \text{ Dhrystones/sec}$. Also, performance of other computers were measured based on VAX 11/780.

3.7 General-Purpose Processor Design

General-purpose processor can be designed using the design technique of single-purpose processor as general-purpose processor is a type of single-purpose processor which process instructions stored in program memory. The design starts with the design of instruction set, followed by creating a FSM and datapath, and finally the controller is developed. All steps are explained in details with example.

A. Instruction Set Design

Instruction set defines various operations that can be done by the processor. It also determines the size of memory required along with number of registers to be used.

- Initially, how many and what kind of operations are to be included must be considered.
- Then, how many, what types, and location of operands to be used must be selected.
- And finally the size and format of instruction must be set.

B. Creating a FSM

FSM represents the state diagram of the given functionality which is based on instruction set. The following steps are required to generate a FSM.

- First, RESET state is defined, which can be used to clear various registers.

- Next, FETCH state is used where instruction from memory is loaded in instruction register. In this state, program counter is increment after each instruction fetch.
- Then, DECODE state is used as a transition state before execution of instructions. In this state, no operation is done but it adds extra cycle necessary for instruction register to get updated.
- Finally, EXECUTE state is defined based on the operation represented by opcode. The number of EXECUTE state is given by the number of instructions available. The operation to be performed is detected just before the start of this state and hence the actual instruction operations are carried out in this state.

C. Building a Datapath

To carry out various operations of FSMD, datapath must be build. The following steps are required.

- For each declared variable, we need to instantiate a storage device
- Then, instantiate functional units to carry out the FSMD operations. General purpose ALU is, usually, implemented in the design.
- Connecting different components within the datapath. When more than one input appear at any ports then appropriate multiplexor is needed.
- Finally, unique identifiers are created for every control signal.

D. Development of Controller or FSM

- Rewrite the FSMD states without any instructions or operations
- Equivalent binary operations on control signals must be written in each state rather than the operation. Each FSMD operation must be replaced by binary operations

EXAMPLE: Design a general purpose processor with four data transfer instruction, two arithmetic operations and one jump instruction.

→ The following are the considerations made in the design.

- 16 bit instruction size, which has direct impact on memory and register selections.
- Instruction Register (IR) and Program Counter (PC) of 16 bit,
- Memory of 64K x 16 bit,
- Register file of 16 x 16 bit

A. Instruction set design

Instruction	First Byte		Second Byte		Operation
MOV Rn, direct	0000	Rn	Direct		$Rn = M(\text{direct})$
MOV direct, Rn	0001	Rn	Direct		$M(\text{direct}) = Rn$
MOV @Rn, Rm	0010	Rn	Rm		$M(Rn) = Rm$
MOV Rn, #imm	0011	Rn	Immediate		$Rn = \text{immediate}$
ADD Rn, Rm	0100	Rn	Rm		$Rn = Rn + Rm$
SUB Rn, Rm	0101	Rn	Rm		$Rn = Rn - Rm$
JZ Rn, relative	0110	Rn	Relative		$PC = PC + \text{relative (if } Rn \text{ is 0)}$

Figure 3.7: A simple instruction set

From the above instruction set, the various means of data transfer and operations can be analyzed which may be useful in developing FSM and datapath.

- The address of memory location are available from
 - Instruction Register: In instruction MOV Rn, direct and MOV direct, Rn, the direct address is used which is available in IR as lower bytes.
 - Register: In instruction MOV @Rn, Rm, the address of memory is given by value of register.
(Address is also given by PC to load the instruction into IR)
- The value in register can be loaded from:
 - Memory: In instruction MOV Rn, direct, register is loaded from memory whose address is given by lower eight bits of IR.
 - Instruction Register: In instruction MOV Rn, #imm, the immediate value of IR is loaded into register.
 - ALU: After execution of ADD Rn, Rm and SUB Rn, Rm, the final result is stored in register.
- Three operations are performed by ALU
 - Addition, subtraction and comparison

B. FSMD for given instruction set

In FSMD, the basic stages of instruction cycle are implemented as states. It includes RESET, FETCH, DECODE and EXECUTE state. The RESET, FETCH and DECODE states are common to almost every design. The EXECUTE state, however, differs when the number and type of instructions are different.

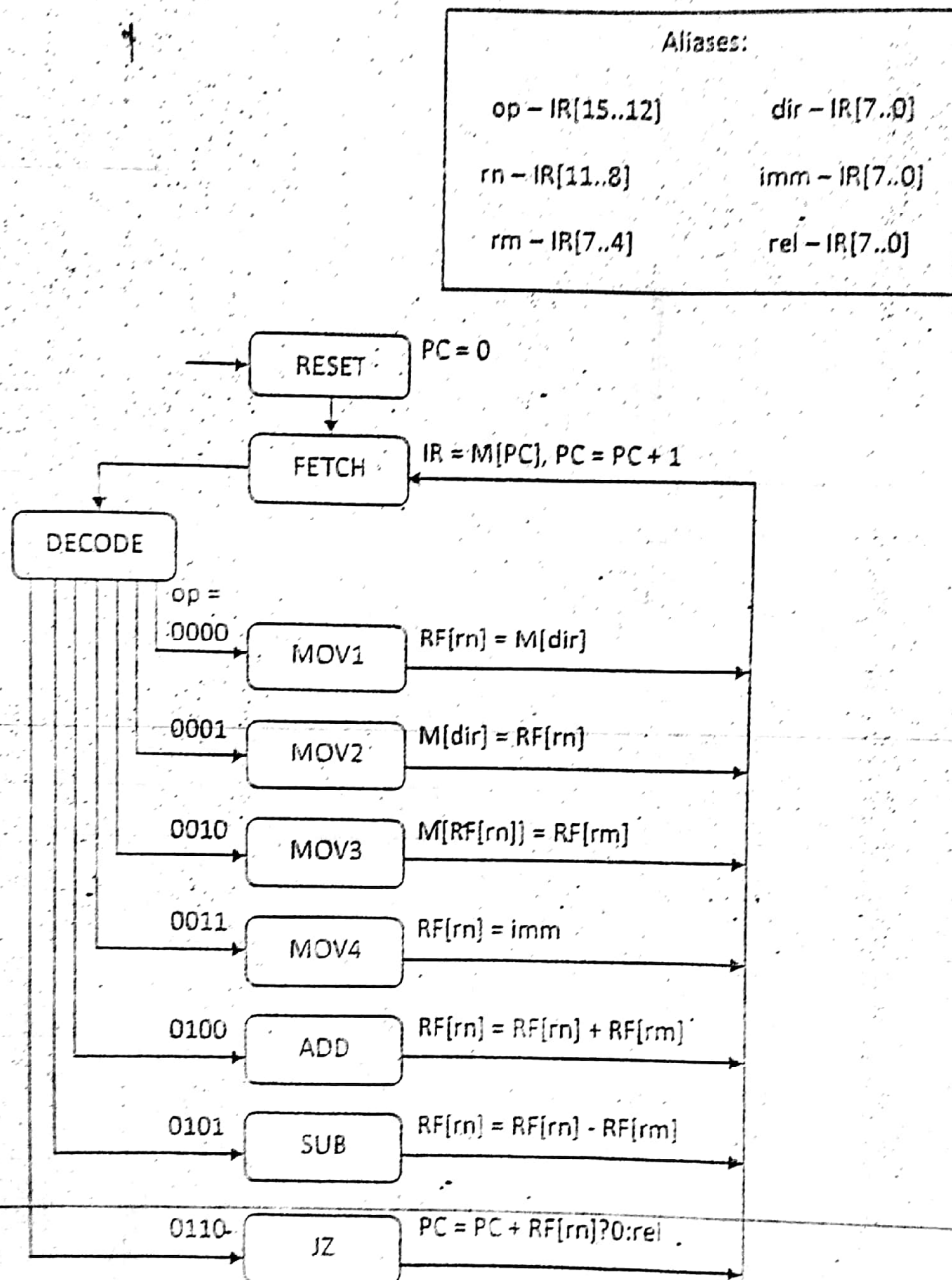


Figure 3.8: Finite State Machine with Data (FSMD)

C. Datapath for FSMD

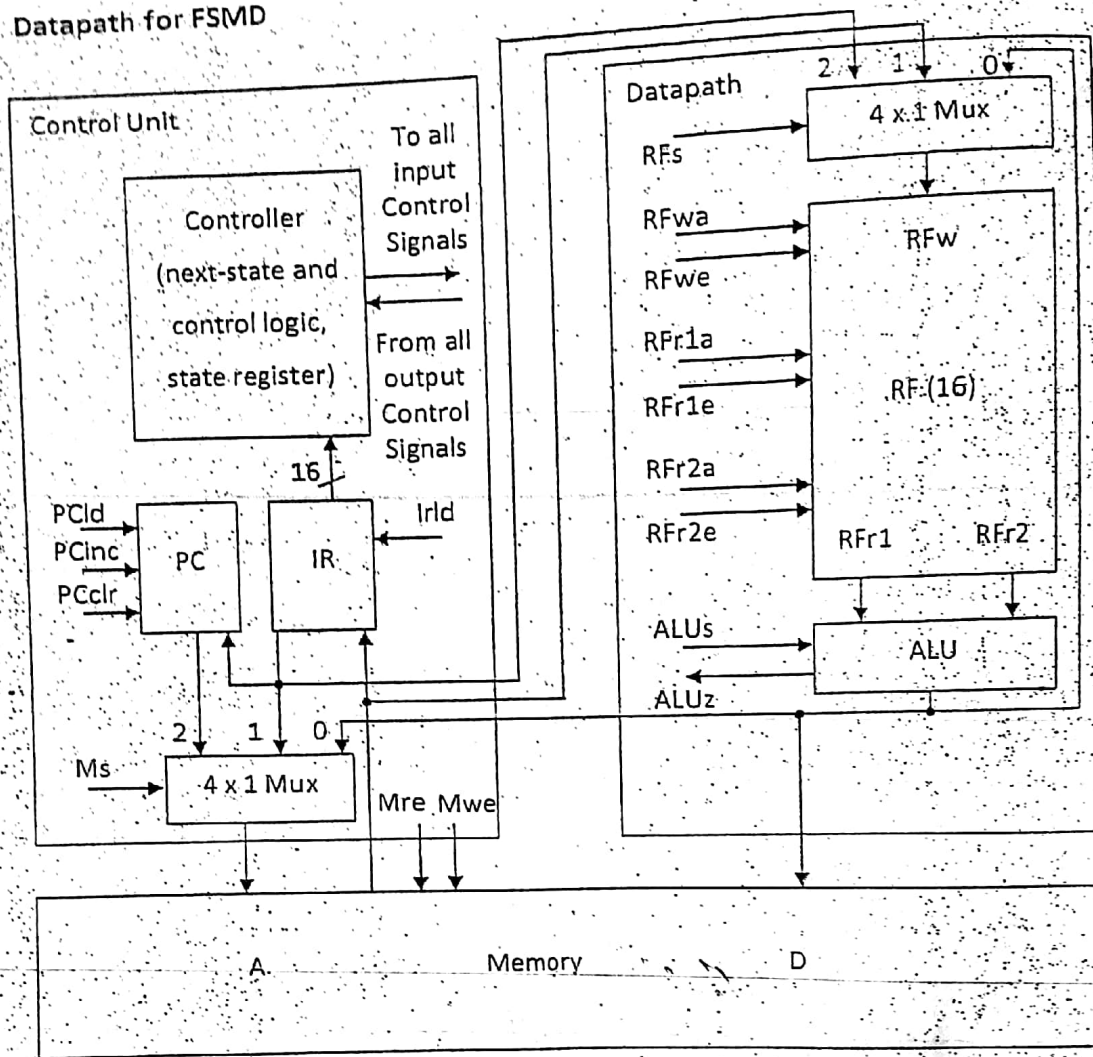


Figure 3.9: Datapath of our simple general purpose processor.

Components in datapath

- Register file of 16x16 and a general purpose ALU.
- Multiplexer of 4x1, since the register in register file can have three sources; Immediate data from IR, data from Memory, and data from ALU

Components in Control Unit

- Controller for next-state and control logic, state register, Program Counter, Instruction Register.
- Multiplexer of 4x1, since memory address can be selected from three sources; address from PC, direct address from IR, and address from register.

D. Finite State Machine (FSM) Design

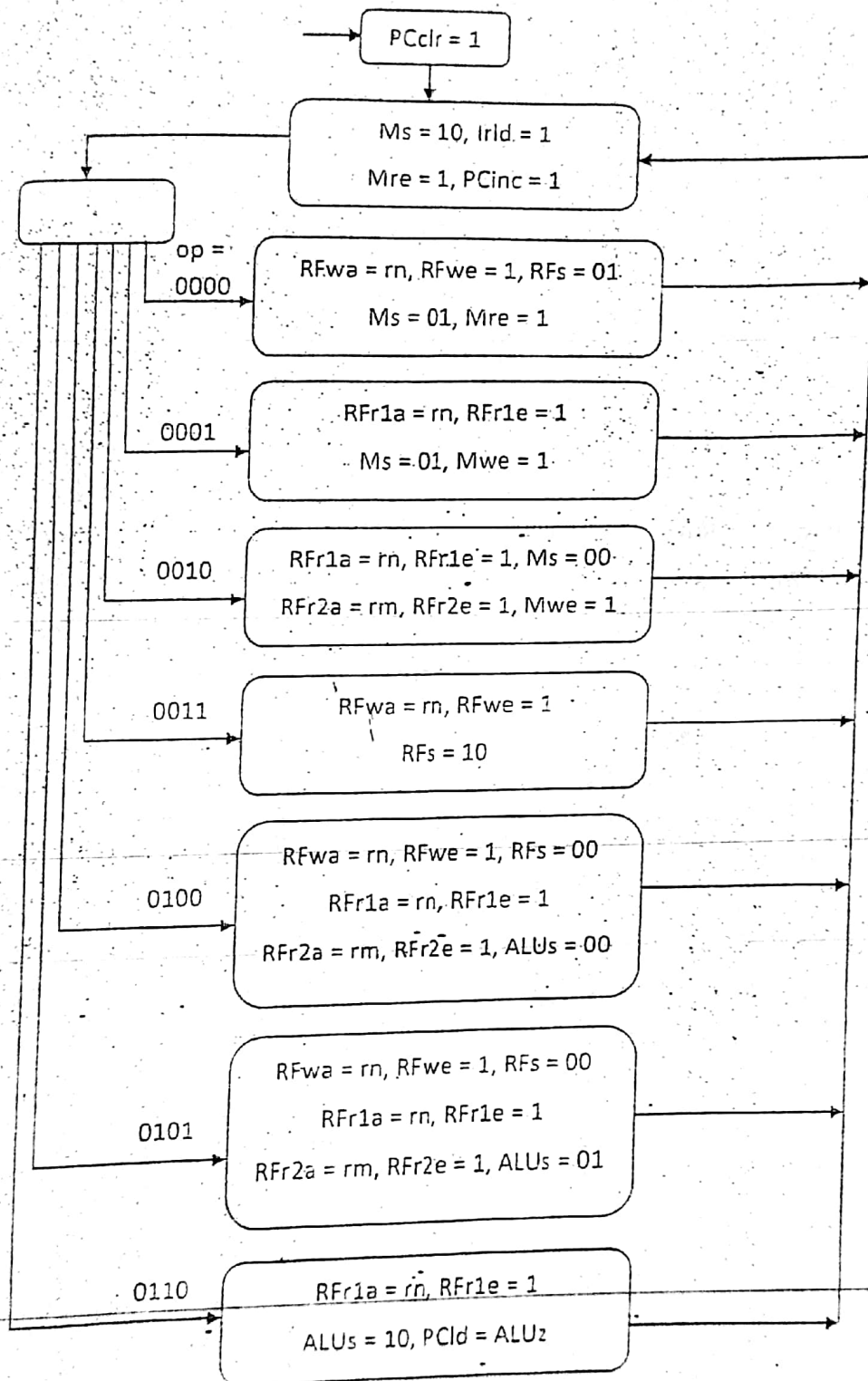


Figure 3.10: Finite State Machine (FSM)

Converting FSMD operations to FSM operations

Example 1: MOV Rn, direct $\rightarrow$ $RF[rn] = M[dir]$

It means to read the content of memory of address dir (8 lower bits of IR) and write it into one of registers of register file. Value of rn gives the address of register in register file.

- Address of memory is directly available in IR, using multiplexer selection: $Ms = 01$ will select address from IR. For a memory read operation, Mre must be set ($Mre = 1$).
- The value is to be written into register file, so $RFwa = rn$ selects a register from register file and $RFwe$ enables the write operation. Set $RFs = 01$, as data is coming from memory.

Example 2: ADD Rn, Rm $\rightarrow$ $RF[rn] = RF[rn] + RF[rm]$

Here, values from two registers are read and then added using ALU. The final result is stored in register. Address of registers to be selected is given by rn and rm for read operation while value of rn gives the address of register for write operation.

- Selection of registers for read operation: $RFr1a = rn$ and $RFr2a = rm$ select two registers while $RFr1e = 1$ and $RFr2e = 1$ enable both registers for read operation.
- Adding the value of registers using ALU: $ALUs = 00$ represent the addition of two registers.
- Selection of register for write operation: $RFwa = rn$ selects the register and $RFwe = 1$ enables the write operation.